Spring boot:Data Jpa

Jpa: ORM Framework (or api),use to perform database operation using
OOPs concept(Object Oriented).
=>. We can perform all operation by writing code.
=>. EntityManager :Has given methods to perform operations persists
[save],merge[updata],find[select], remove[delete]

Data JPA: It will generate code to perform operations.
=> This is given by spring boot,it internally generates JPA code only.
Proxy: A class created at runtime and object also.

A Proxy is a design pattern in which an object acts as an intermediary between the client and the real object.It controls access to the real object.

Real-life example
A security guard stands between a visitor and a restricted area. The guard checks permission before allowing entry.

Working
1. The client requests a service from the Proxy.
2. The Proxy performs checks or other additional work.
3. If appropriate, the Proxy forwards the request to the real object.
4. The real object performs the actual task.

Flow: Client → Proxy → Real Object

```
class ProxyInternet implements Internet {

private RealInternet realInternet = new RealInternet();

public void connect() {

System.out.println("Checking access...");

// Forward the request to the real object
realInternet.connect();
}
}
```

```
class RealInternet implements Internet {
    public void connect() {
        System.out.println("Internet connected!");
    }
}
```

Why use Proxy?
- Control access to an object.
- Check permissions.
- Delay expensive object creation.
- Cache results.
- Handle access to remote objects.

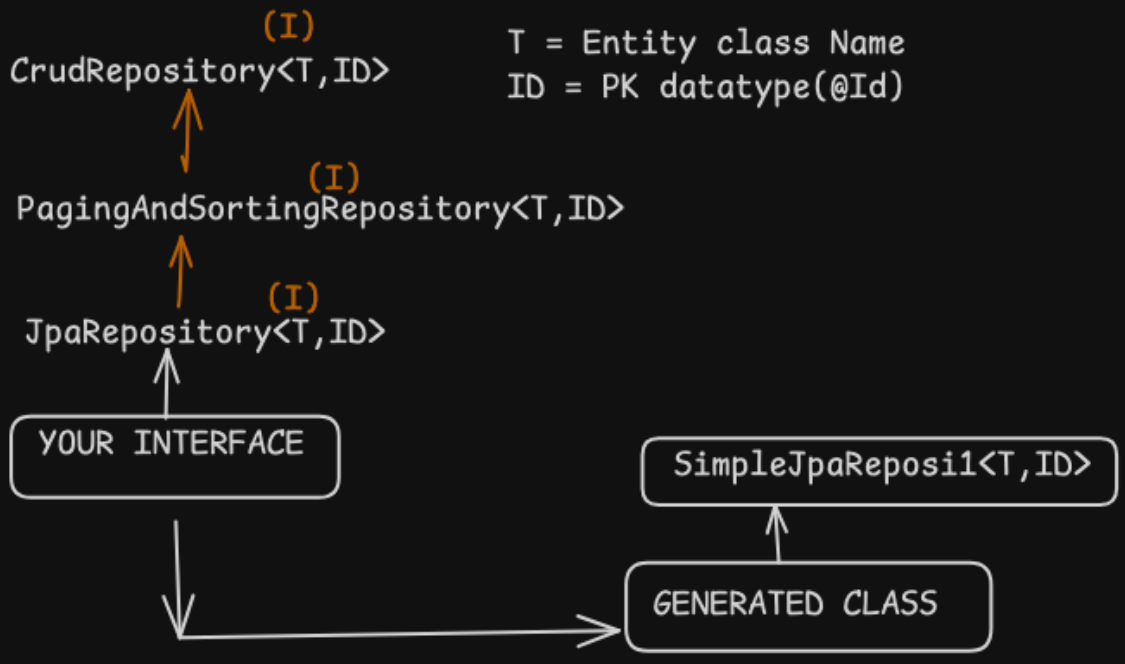
Types
- Protection Proxy: controls permissions.
- Virtual Proxy: delays object creation.
- Remote Proxy: represents a remote object.
- Caching Proxy: reuses stored results.

Proxy in Spring Boot
Spring commonly uses proxies to support features such as `@Transactional`, method-level security, and caching.

Key point
The Proxy provides an additional layer of control without requiring the client to directly access the real object.

--Given repository by spring JPA----

CrudRepository<T,ID>: {Insert,delete,select,update}
PagingAndSortingRepository<T,ID>: {Sorting,pagination}
JpaRepository<T,ID> : {JPA based operation.,ex-> obj,batch,..etc}



SimpleJpaRepository<T,ID> : It is class having internally JPA code for above interface
=> SimpleJpaRepository is a template class that is how Database operation code look like {generic code, not for specific class}
=> we need to define one interface that extends any one of the Repetitive interface
=> Internally data JPS Uses simple JPA repository and - repository interface and generate code One class that is called proxy

Q=> What is DAO and DTO ?
Ans DAO: data access object
A class that contain code to perform database operations.
DTO: A class contains(variables,set/get)passed between method
Q=> entity vd DTO ?
Ans Entity: a class mapped with database table.
DTO: a class contain Variables set/get, used to pass as object from one method to another method.
Q=> what is Respository ?
Ans An Interface define with type and ID that is used to perform database operation.
Q=> what is dynamic proxy?

Ans A class /code class. generated at runtime generated at runtime by taking set of interface and a template(formate) class also, some inputs(T,ID)

Code

```
spring.application.name=Project_2
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=Gaurav@123

spring.jpa.show-sql=true
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=create
```

```
package com;

import org.springframework.data.jpa.repository.JpaRepository;
public interface StudentRepo extends JpaRepository<student, Integer> {

}
```

```
package com;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.Data;

@Data
@Entity
public class student {
    @Id
    private Integer id;
    private String name;
}
```

```
package com;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class testrunner implements CommandLineRunner {
	@Autowired
	private StudentRepo repo;
	@Override
	public void run(String... args) throws Exception {
		// TODO Auto-generated method stub
		student s=new student();
		s.setId(10);
		s.setName("Rahul");
		repo.save(s);
	}

}
```

In hibernate we write all the transaction/session/ and many more thing but here we are not write even a simple code related to this

This is example when i write through hibernate →

```
package com.example.demo;

import org.hibernate.Transaction;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import com.mysql.cj.Session;
import com.mysql.cj.xdevapi.SessionFactory;

import jdk.jfr.internal.query.Configuration;

@SpringBootApplication
public class Project1Application {

	public static void main(String[] args) {
		SpringApplication.run(Project1Application.class, args);
		org.hibernate.SessionFactory fact=new
org.hibernate.cfg.Configuration().configure().buildSessionFactory();

		first f=new first();
		f.setId(2);
		f.setName("Gaurav");
		org.hibernate.Session session=fact.openSession();
		Transaction tx=session.beginTransaction();
		session.save(f);
		tx.commit();
		session.close();
		System.out.println("all completed");
	}

}
```

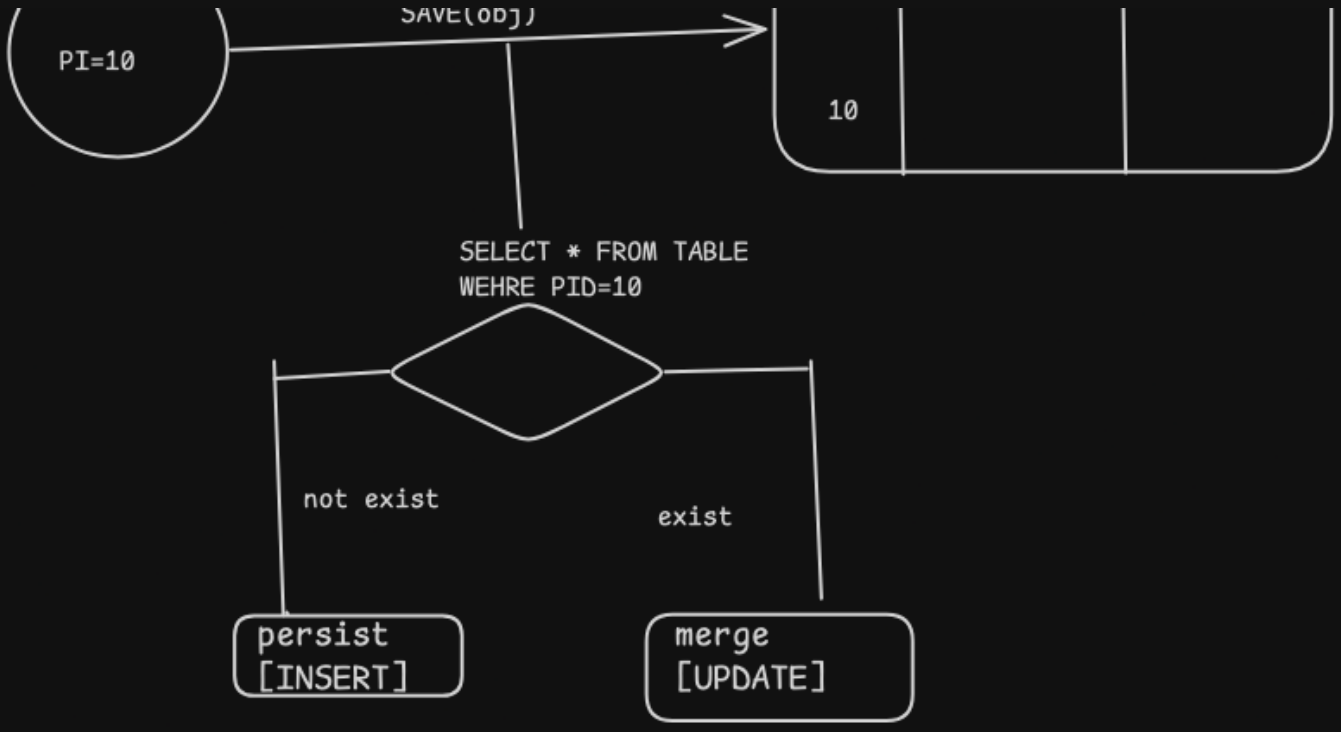
CrudRepository

1. save(Obj) :obj
   This method will check given object is present or not. Based on Primary key inside a DB table .

=> SELECT * FROM TABLE WHERE PK_COL=VAL
=> if record is present -> then update now
=> Else INSERT ROW
* if same object data is present in Db, then save() method does

nothing {No update or no insert}





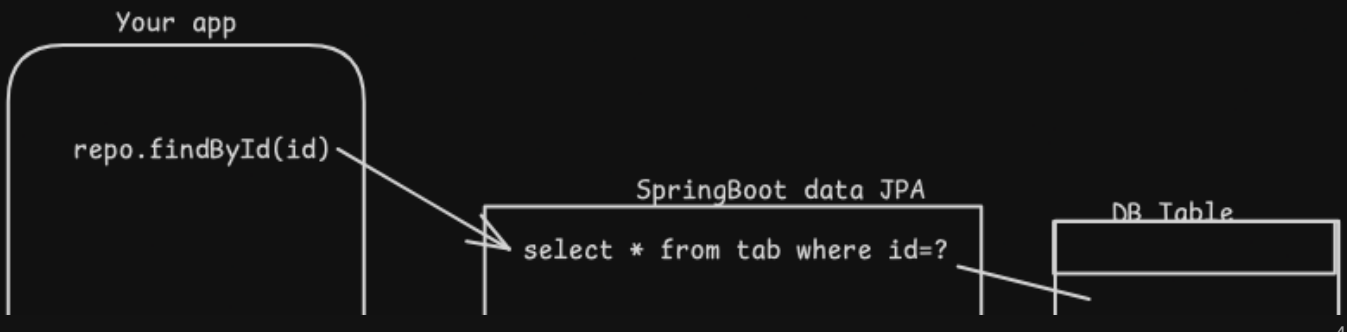
2. saveAll(Iterable) ->
   This method takes multiple objects at a time as a collection and perform same opertion line save() method.
   ex: repo.saveAll(Array.asList(p1,p2,p3));
3. findAll(Iterable) ->
   This method is used to fetch all rows from DB table.Gives as Iterable formate.

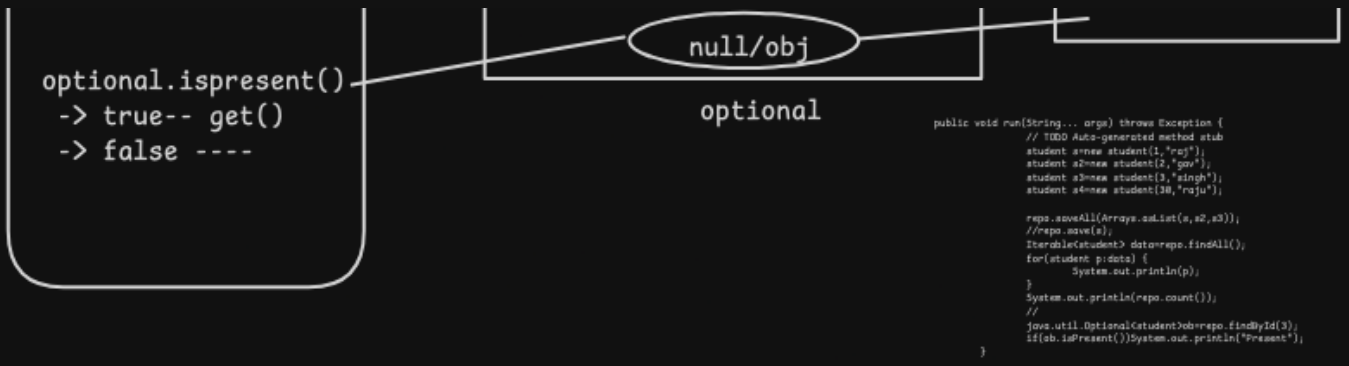
   SQL: select * form

4. existsById. ->
   it will check given ID (row) present in db table or not.
   if present then true ,else false.it internally use count query.

5. count() :long
   it count how may








```
public void run(String... args) throws Exception {
		// TODO Auto-generated method stub
		student s=new student(1,"raj");
		student s2=new student(2,"gov");
		student s3=new student(3,"singh");
		student s4=new student(30,"raju");

		repo.saveAll(Arrays.asList(s,s2,s3));
		//repo.save(s);
		Iterable<student> data=repo.findAll();
		for(student p:data) {
			System.out.println(p);
		}
		System.out.println(repo.count());
		//
		java.util.Optional<student>ob=repo.findById(3);
		if(ob.isPresent())System.out.println("Present");
	}
```

JDK 1.8 :Optional<T> :This is used to avoid null problems
[NullPointerException only]

=> It is like a wrapper object to the actual data.
(or) container object for actual data.

=> Optional returns Actual data if present (isPresent() true)
or else it return empty data.

6. findById(id) : Optional<T>
this is the method is used to fetch one row data from DB based on PK.
SQL: select * from product where id=10

==========

Q. Explain here what is X And ? Here.

```
<X extends Throwable >
Supplier<? extends X >
```

Ans-> Here X can be any subclass of Throwable(Direct or Indirect)
? (Any Type of give/wild card character).
it can be any sub type of X (or even same type)

Q. Explain below line ?

```
public static <T, K,U> Collector<T, ? ,Map<K,U>>toMap(
   Function<? super T,? extends K> keyMapper,
        Function<? super T,? extends U > valueMapper)
```

Ans. A Collector tells Java how to collect elements from a Stream
create a final result.

Java provides Collectors.toMap() to ->.You want to convert this List into a Map.

What is T, K, U? ->.These are called Generic Type Parameters.
Think of them as temporary names. We don't know the actual data types yet.

Function is a Java Functional Interface.-> Function takes one input and returns one output.

```
Function<String, Integer> getLength = name->name.length();

getLength.apply("Rahul");
```

Input, Output, To call

Syntax	Meaning	Main use
? extends T	T or its subclass	Read
? super T	T or its superclass	Add

Function<? super T, ? extends K> ->.

? super T. =>
Means the Function can accept an input of type T or one of its parent types

? extends K. =>
Means the Function can return a value of type K or a child type of K.

A function that accepts a T-compatible input and returns a K-compatible output.

A Collector tells Java how to collect elements from a Stream and create a final result.

Collector<T, ?, Map<K, U>>
Meaning:
- T → Type of input elements.
- ? → Internal temporary storage type used while collecting. You don't need to specify it here.

- Map<K,U> → Final result will be a Map.

```
public static void main(String[] args) {

        List<String> students =
            List.of("Rahul", "Aman", "Gaurav");

        Map<String, Integer> result =
            students.stream()
                .collect(Collectors.toMap(
                    name -> name,
                    name -> name.length()
                ));

        System.out.println(result);
    }
```